MATERIAL SEMANA 02

JSX, Componentes, Props e Eventos

Instituição: SENAI-SP - Serviço Nacional de Aprendizagem Industrial
Curso: Técnico em Desenvolvimento de Sistemas
Unidade Curricular: Desenvolvimento de Frameworks - React
Semana: 02
Projeto: To-Do Pro - Gestão de tarefas fullstack
Carga sugerida: 3h45min
Nível: Fundamentação e modularização prática de interfaces

Roteiro de estudos da semana

Este material foi planejado para guiar seu estudo autônomo e prático no Classroom. O percurso está organizado em quatro movimentos essenciais:

1. **Compreender:** entenda por que dividimos uma aplicação em componentes e como dados e eventos trafegam entre eles.
2. **Modularizar:** crie componentes especializados com responsabilidade única na pasta `src/components/`.
3. **Interagir e Iterar:** conecte propriedades (*props*), manipule eventos (*onClick*, *onChange*) e gere listas dinâmicas com `map()` e chaves únicas (`key`).
4. **Entregar:** estilize a aplicação de forma responsiva, execute testes manuais e registre sua evolução com commits semânticos no Git.

---

1. Como estudar este material

Para aproveitar ao máximo este guia:

1. **Abra seu editor e digite os exemplos:** evite copiar e colar. Digitar ajuda a fixar a sintaxe do JSX e das funções.
2. **Observe o console do navegador (F12):** mensagens em amarelo (avisos) e vermelho (erros) são suas melhores professoras. Aprenda a lê-las.
3. **Faça os exercícios na ordem proposta:** cada bloco prepara você para o passo seguinte.
4. **Mantenha funções curtas:** no máximo 20 linhas por função e com foco em uma única responsabilidade (SOLID).
5. **Salve e teste com frequência:** mantenha o servidor do Vite rodando (`npm run dev`) para ver as mudanças em tempo real.

---

2. O desafio do To-Do Pro na Semana 02

Na Semana 01, você colocou no ar a primeira versão da aplicação **To-Do Pro**, concentrando todo o código no arquivo `App.jsx`.

Agora, a empresa contratante solicitou uma evolução importante:

- As tarefas precisam ser exibidas individualmente em cartões com título, descrição, prioridade e status;
- Cada tarefa deve conter botões para concluir ou remover;
- Quando não houver tarefas cadastradas, a tela deve exibir uma mensagem amigável de lista vazia;
- O sistema deve carregar uma lista simulada de tarefas a partir de um arquivo de dados (`tasksMock.js`).

Manter tudo isso em um único arquivo tornaria o código confuso e impossível de manter. O objetivo desta semana é **refatorar** a aplicação para uma arquitetura profissional de componentes reutilizáveis.

---

3. Objetivos de aprendizagem

Ao concluir este material, você será capaz de:

- decompor uma tela em componentes funcionais com responsabilidade única;
- organizar os arquivos do projeto dentro de `src/components/`;
- passar informações do componente pai para componentes filhos através de *props*;
- utilizar desestruturação e parâmetros padrão em componentes React;
- compreender e respeitar a regra de imutabilidade das *props*;
- capturar eventos do usuário (`onClick`, `onChange`, `onSubmit`) no JSX;

- comunicar ações de um componente filho para o pai usando funções *callback*;
- renderizar coleções de dados com `Array.prototype.map()`;
- utilizar a propriedade `key` única e entender seu papel no Virtual DOM;
- implementar renderização condicional com operadores ternários e curto-circuito (`&&`);
- criar um layout limpo e responsivo com CSS Grid e Media Queries;
- documentar e versionar as entregas com Git.

---

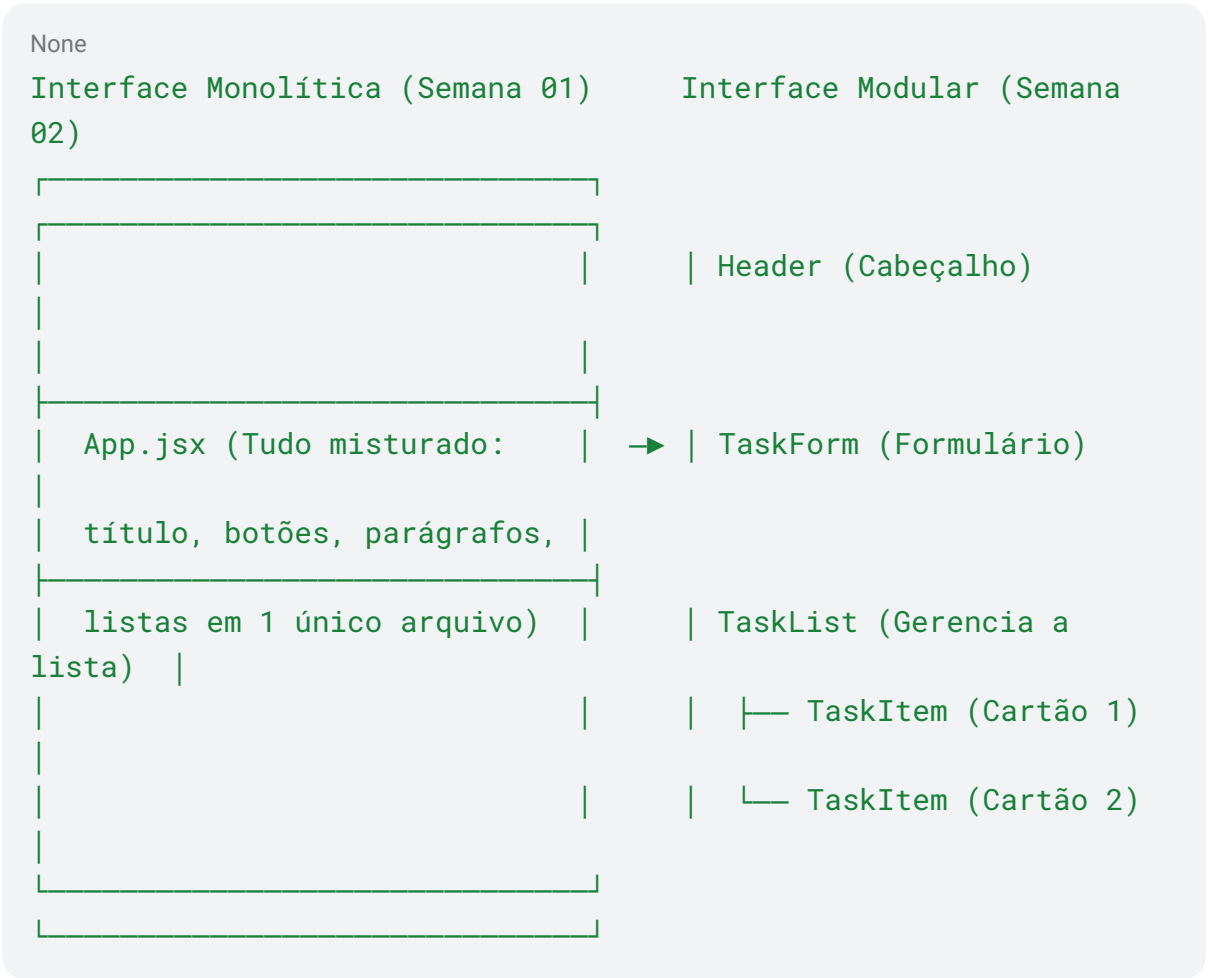
4. Conceitos fundamentais

4.1 Por que componentizar?

Pense em um carro: ele não é uma peça única de ferro fundido. Ele é composto por motor, rodas, bancos, painel e portas. Se um pneu fura, você troca apenas o pneu, sem precisar reconstruir o motor.

No React, a interface segue a mesma lógica. Um **componente** é um bloco de código independente, reutilizável e isolado, responsável por desenhar uma parte da tela e controlar seu comportamento.

None



4.2 O Princípio da Responsabilidade Única (SRP)

O princípio da responsabilidade única diz que **cada componente deve ter apenas um motivo para mudar**:

- Se mudarmos o logotipo ou subtítulo da empresa, apenas o componente `Header` é alterado;
- Se adicionarmos um novo botão de ação na tarefa, apenas `TaskActions` muda;
- Se a forma de exibir o cartão da tarefa mudar, apenas `TaskItem` é modificado.

4.3 O que são Props?

Se um componente é uma função JavaScript, as **props** (abreviação de *properties*) são os **argumentos** dessa função.

Props permitem que o componente pai envie dados para seus filhos:

```
None
// O pai decide o que exibir passando propriedades
<TaskItem title="Estudar React" priority="Alta" />
```

```
None
// O filho recebe os dados e os apresenta na tela
function TaskItem(props) {
  return <h3>{props.title} - Prioridade: {props.priority}</h3>;
}
```

4.4 Fluxo unidirecional de dados e imutabilidade

No React, os dados trafegam exclusivamente em sentido único: **de cima para baixo (do pai para o filho)**.

Regra de Ouro: Props são somente leitura!

Um componente **nunca deve alterar suas próprias props**.

```
None
// NUNCA faça isso:
function TaskItem(props) {
  props.title = "Outro título"; // ERRO! Quebra o React.
```

```
}
```

Se os dados precisam mudar dinamicamente ao longo do tempo, veremos na Semana 03 que isso é responsabilidade do **Estado** (`state`).

4.5 Desestruturação e valores padrão

Em vez de escrever `props.title`, `props.description` e `props.priority`, podemos desestruturar o objeto diretamente nos parâmetros da função e já definir valores padrão caso a propriedade não seja enviada:

```
None
// src/components/TaskItem.jsx
// Recebe propriedades desestruturadas com prioridade padrao
function TaskItem({ title, description, priority = "Normal" }) {
  return (
    <article className="task-item">
      <h3>{title}</h3>
      <p>{description}</p>
      <span className="badge">{priority}</span>
    </article>
  );
}

export default TaskItem;
```

Exercício 1 - Praticando Props e Valores Padrão

Crie mentalmente ou em um arquivo de rascunho um componente chamado `UserBadge`:

1. Ele deve receber `name` e `role`.
2. Se `role` não for informada, o valor padrão deve ser `"Estudante"`.
3. O componente deve retornar:

```
None
<div className="badge">
  <strong>{name}</strong> - <span>{role}</span>
</div>
```

Verificação: ao instanciar `<UserBadge name="Carlos" />`, o navegador deve exibir `Carlos - Estudante`.

4.6 Eventos sintéticos no React

No HTML comum, usamos atributos minúsculos com strings: `<button onclick="executar()">`.

No React, usamos **camelCase** e passamos **uma função real**:

```
None
// Passando a referencia da funcao manipuladora
<button type="button" onClick={handleClick}>
  Clique aqui
</button>
```

A armadilha da chamada imediata

Observe com muita atenção a diferença entre estes três formatos:

Sintaxe no JSX	O que acontece?	Quando usar?
`onClick={handleClick}`	Passa a referência da função. O React só a executa quando o usuário clica.	Quando a função **não precisa** de parâmetros.
`onClick={() => handleClick(task.id)}`	Cria uma arrow function que só roda no clique, passando o id.	Quando você **precisa passar parâmetros** na chamada.
`onClick={handleClick()}`	**ERRADO!** Executa a função imediatamente na montagem da tela!	Quase nunca! Gera loops e disparos indesejados.

4.7 Comunicação Filho-para-Pai via Callbacks

Se os dados só descem dos pais para os filhos, como um botão no componente `TaskItem` avisa o `App` que deve remover uma tarefa?

O segredo é a **passagem de funções como props (callbacks)**:

1. O pai (`App`) cria a função de remover;
2. O pai passa essa função para o filho: `<TaskItem onRemove={minhaFuncaoDeRemover} />`;
3. No clique do botão, o filho dispara a função que pertence ao pai!

```
None
[App.jsx]
   |
   | 1. Passa callback: onRemoveTask={handleRemove}
   ▼
[TaskItem.jsx]
   |
   | 2. O usuário clica no botão "Remover"
   ▼
[TaskActions.jsx] ——> Dispara: onRemove() ——> [Executa no
App.jsx!]
```

4.8 Listas dinâmicas com `map()` e a prop `key`

No JavaScript moderno, o método `.map()` transforma os itens de um array em elementos visuais JSX:

```
None
const frutas = ["Maçã", "Banana", "Laranja"];

return (
  <ul>
    {frutas.map((fruta, index) => (
      <li key={index}>{fruta}</li>
    ))}
  </ul>
);
```

Por que o React exige a prop `key`?

O React usa uma cópia virtual da tela chamada **Virtual DOM**. Quando um item é adicionado, reordenado ou excluído de uma lista, o React precisa saber com exatidão **qual elemento mudou** para atualizar apenas ele no navegador.

Atenção com chaves baseadas em índice!

Evite usar `key={index}` em listas onde tarefas podem ser excluídas ou reordenadas. Use sempre um identificador único fixo do objeto, como `key={task.id}`.

4.9 Renderização condicional

Muitas vezes a tela precisa exibir conteúdos diferentes dependendo da situação:

1. Operador ternário (`condicao ? valorSeVerdade : valorSeFalso`):

```
None
// Mostra mensagem diferente conforme o status
<p>{completed ? "Tarefa Concluída!" : "Tarefa Pendente."}</p>
```

2. Curto-circuito lógico (`condicao && elemento`):

```
None
// So renderiza a tag se a tarefa for prioritária
{isUrgent && <span className="tag-alerta">URGENTE</span>}
```

3. Retorno antecipado (Early Return):

```
None
// Se o array estiver vazio, encerra e retorna o aviso
if (tasks.length === 0) {
  return <p className="empty-msg">Nenhuma tarefa encontrada.</p>;
}
```

---

5. Passo a passo prático: construindo o To-Do Pro modular

Agora vamos construir a versão modular da Semana 02 no seu projeto!

Passo 1: Organizar as pastas do projeto

Abra a pasta do projeto no VS Code e certifique-se de criar os diretórios `src/components/` e `src/data/`:

```
None
src/
├── components/
|   ├── Header.jsx
```

```
|   ├── TaskActions.jsx
|   ├── TaskForm.jsx
|   ├── TaskItem.jsx
|   └── TaskList.jsx
├── data/
|   └── tasksMock.js
├── App.css
├── App.jsx
└── main.jsx
```

Passo 2: Criar o arquivo de dados simulados (`src/data/tasksMock.js`)

Crie o arquivo `src/data/tasksMock.js` com uma lista inicial de tarefas:

JavaScript

```javascript
// src/data/tasksMock.js
// Dados simulados de tarefas para desenvolvimento da interface
export const initialTasks = [
  {
    id: "task-01",
    title: "Configurar o ambiente React",
    description: "Instalar Node.js LTS, VS Code e extensões recomendadas.",
    priority: "Alta",
    completed: true,
  },
  {
    id: "task-02",
    title: "Modularizar componentes da interface",
    description: "Separar Header, TaskForm, TaskList e TaskItem em arquivos próprios.",
    priority: "Alta",
    completed: false,
  },
  {
    id: "task-03",
```

```
    title: "Praticar passagem de props e eventos",
    description: "Transmitir dados do pai para os filhos e testar
callbacks de clique.",
    priority: "Normal",
    completed: false,
  },
  {
    id: "task-04",
    title: "Estilizar layout responsivo",
    description: "Ajustar regras com CSS Grid e Media Queries
para celular e PC.",
    priority: "Baixa",
    completed: false,
  },
];
```

Passo 3: Criar o componente `Header.jsx`

Crie `src/components/Header.jsx`:

JavaScript

```
// src/components/Header.jsx
// Exibe o titulo e o subtitulo principal da aplicacao
function Header() {
  return (
    <header className="app-header">
      <h1>To-Do Pro</h1>
      <p>Gerenciamento inteligente de tarefas com React.</p>
    </header>
  );
}

export default Header;
```

Passo 4: Criar o componente de botões `TaskActions.jsx`

Crie `src/components/TaskActions.jsx`:

JavaScript

```
// src/components/TaskActions.jsx
// Componente de botoes que emite acoes via funcoes callback
function TaskActions({ completed, onToggle, onRemove }) {
  return (
    <div className="task-actions">
      <button
        type="button"
        className={completed ? "btn-reopen" : "btn-done"}
        onClick={onToggle}
      >
        {completed ? "Reabrir" : "Concluir"}
      </button>

      <button type="button" className="btn-delete"
onClick={onRemove}>
        Remover
      </button>
    </div>
  );
}

export default TaskActions;
```

Passo 5: Criar o cartão de tarefa `TaskItem.jsx`

Crie `src/components/TaskItem.jsx`:

JavaScript

```
// src/components/TaskItem.jsx
import TaskActions from "./TaskActions";

// Cartao de tarefa que recebe dados por props e delega acoes
function TaskItem({ id, title, description, priority = "Normal",
completed, onToggle, onRemove }) {
  const cardClass = `task-card ${completed ? "is-completed" :
""}`;
```

```
  return (
    <article className={cardClass}>
      <div className="task-info">
        <header className="task-card-header">
          <h3>{title}</h3>
          <span className={`priority-tag
priority-${priority.toLowerCase()}`}>
            {priority}
          </span>
        </header>
        <p className="task-description">{description}</p>
        <small className="task-status">
          Status: {completed ? "✓ Concluída" : "⏳ Pendente"}
        </small>
      </div>

      <TaskActions
        completed={completed}
        onToggle={() => onToggle(id)}
        onRemove={() => onRemove(id)}
      />
    </article>
  );
}

export default TaskItem;
```

Passo 6: Criar o componente de lista `TaskList.jsx`

Crie `src/components/TaskList.jsx`:

JavaScript

```
// src/components/TaskList.jsx
import TaskItem from "./TaskItem";

// Renderiza a grade de tarefas ou mensagem informativa de lista
vazia
```

```
function TaskList({ tasks = [], onToggleTask, onRemoveTask }) {
  if (tasks.length === 0) {
    return (
      <div className="empty-container">
        <p className="empty-message">Nenhuma tarefa encontrada no
momento.</p>
      </div>
    );
  }

  return (
    <section className="task-list">
      {tasks.map((task) => (
        <TaskItem
          key={task.id}
          id={task.id}
          title={task.title}
          description={task.description}
          priority={task.priority}
          completed={task.completed}
          onToggle={onToggleTask}
          onRemove={onRemoveTask}
        />
      ))}
    </section>
  );
}

export default TaskList;
```

Passo 7: Criar o formulário inicial `TaskForm.jsx`

Crie `src/components/TaskForm.jsx`:

JavaScript

```
// src/components/TaskForm.jsx
// Formulario acessivel para entrada de tarefas com prevencao de
recarregamento
```

```
function TaskForm({ onAddTask }) {
  function handleSubmit(event) {
    event.preventDefault();
    onAddTask("Nova tarefa adicionada via formulário");
  }

  return (
    <form className="task-form" onSubmit={handleSubmit}>
      <label htmlFor="task-input">Nova tarefa</label>
      <div className="form-row">
        <input
          id="task-input"
          type="text"
          placeholder="Ex.: Revisar código do To-Do Pro..."
          required
        />
        <button type="submit" className="btn-primary">
          Cadastrar
        </button>
      </div>
    </form>
  );
}

export default TaskForm;
```

Passo 8: Integrar tudo no `src/App.jsx`

Substitua o conteúdo de `src/App.jsx`:

JavaScript

```
// src/App.jsx
import Header from "./components/Header";
import TaskForm from "./components/TaskForm";
import TaskList from "./components/TaskList";
import { initialTasks } from "./data/tasksMock";
import "./App.css";
```

```
// Componente principal agregador do To-Do Pro
function App() {
  function handleToggleTask(taskId) {
    console.log("Ação solicitada: alternar status da tarefa:",
taskId);
  }

  function handleRemoveTask(taskId) {
    console.log("Ação solicitada: remover tarefa:", taskId);
  }

  function handleAddTask(newTaskTitle) {
    console.log("Ação solicitada: cadastrar tarefa:",
newTaskTitle);
  }

  return (
    <main className="app-container">
      <Header />
      <TaskForm onAddTask={handleAddTask} />
      <TaskList
        tasks={initialTasks}
        onToggleTask={handleToggleTask}
        onRemoveTask={handleRemoveTask}
      />
    </main>
  );
}

export default App;
```

Passo 9: Estilizar com CSS Responsivo (`src/App.css`)

Adicione ou substitua o conteúdo em `src/App.css`:

CSS

```
/* src/App.css */
* {
```

```
  box-sizing: border-box;
}

body {
  margin: 0;
  background-color: #f1f5f9;
  color: #1e293b;
  font-family: system-ui, -apple-system, sans-serif;
}

.app-container {
  max-width: 860px;
  margin: 2rem auto;
  padding: 0 1rem;
}

.app-header {
  text-align: center;
  margin-bottom: 2rem;
}

.app-header h1 {
  margin: 0;
  color: #0f172a;
}

.task-form {
  background: #ffffff;
  padding: 1.25rem;
  border-radius: 8px;
  box-shadow: 0 1px 3px rgba(0,0,0,0.1);
  margin-bottom: 1.5rem;
}

.form-row {
  display: flex;
  gap: 0.5rem;
  margin-top: 0.5rem;
}
```

```css
.form-row input {
  flex: 1;
  padding: 0.6rem;
  border: 1px solid #cbd5e1;
  border-radius: 6px;
  font-size: 1rem;
}

.btn-primary {
  background-color: #2563eb;
  color: #ffffff;
  border: none;
  padding: 0.6rem 1.2rem;
  border-radius: 6px;
  cursor: pointer;
  font-weight: 600;
}

/* Lista de Tarefas em Grade */
.task-list {
  display: grid;
  grid-template-columns: 1fr;
  gap: 1rem;
}

/* Responsividade: duas colunas em telas a partir de 640px */
@media (min-width: 640px) {
  .task-list {
    grid-template-columns: repeat(2, 1fr);
  }
}

.task-card {
  background: #ffffff;
  border: 1px solid #e2e8f0;
  border-radius: 8px;
  padding: 1rem;
  display: flex;
```

```
  flex-direction: column;
  justify-content: space-between;
}

.task-card.is-completed {
  background-color: #f8fafc;
  opacity: 0.75;
  border-left: 5px solid #10b981;
}

.task-card-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.task-card-header h3 {
  margin: 0;
  font-size: 1.1rem;
}

.priority-tag {
  font-size: 0.75rem;
  padding: 0.2rem 0.5rem;
  border-radius: 4px;
  font-weight: 600;
}

.priority-alta { background: #fee2e2; color: #dc2626; }
.priority-normal { background: #e0f2fe; color: #0284c7; }
.priority-baixa { background: #f3f4f6; color: #4b5563; }

.task-actions {
  display: flex;
  gap: 0.5rem;
  margin-top: 1rem;
}

.btn-done, .btn-reopen, .btn-delete {
```

```
  flex: 1;
  padding: 0.4rem;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  font-size: 0.85rem;
  font-weight: 500;
}

.btn-done { background: #dcfce7; color: #15803d; }
.btn-reopen { background: #fef9c3; color: #a16207; }
.btn-delete { background: #fee2e2; color: #b91c1c; }

.empty-container {
  text-align: center;
  padding: 2rem;
  background: #ffffff;
  border-radius: 8px;
}
```

---

6. Guia de erros e soluções comuns

1. Aviso: *Each child in a list should have a unique "key" prop*

- **Causa:** Você utilizou `.map()` para listar tarefas, mas não colocou a prop `key={task.id}` no elemento mais externo retornado dentro da iteração.
- **Como corrigir:** Certifique-se de que a tag raiz dentro do `.map()` possui `key={task.id}`:

None
```
{tasks.map((task) => (
  <TaskItem key={task.id} {...task} />
))}
```

2. A função executa sozinha assim que abro a página!

- **Causa:** Você colocou parênteses no `onClick`: `onClick={minhaFuncao()}`.

- **Como corrigir:** Remova os parênteses `onClick={minhaFuncao}` ou use uma arrow function se precisar passar parâmetros: `onClick={() => minhaFuncao(id)}`.

3. Erro: *Adjacent JSX elements must be wrapped in an enclosing tag*

- **Causa:** O componente tentou retornar duas tags irmãs sem um elemento pai.
- **Como corrigir:** Envolva os elementos em uma tag única (`<div>...</div>`) ou em um Fragment vazio (`<>...</>`).

4. Erro: *Failed to resolve import "./components/Header"*

- **Causa:** O nome do arquivo na pasta não coincide com o nome na importação (lembre-se que o Linux e o Git diferenciam maiúsculas e minúsculas).
- **Como corrigir:** Verifique se o arquivo chama-se `Header.jsx` e está dentro de `src/components/`.

---

7. Desafio de entrega da semana: To-Do Pro v2

Critérios de Aceite da Entrega:

Interface decomposta em pelo menos 4 componentes dentro de `src/components/`;
Todas as funções com no máximo 20 linhas e comentários em `pt-BR`;
Dados carregados a partir do mock `src/data/tasksMock.js`;
Nenhum aviso de `key` no console do navegador;
Botões de "Concluir" e "Remover" exibindo mensagens de log com o `id` da tarefa no console;
Layout responsivo testado em tela de smartphone (coluna única) e computador (duas colunas);
Pelo menos dois commits semânticos registrados no Git.

Comandos para versionar sua entrega no terminal:

```
None
git status
git add .
```

```
git commit -m "feat: implementa componentes modulares com props, eventos e listas"
git log --oneline -2
```

---

8. Tabela de autoavaliação

Avalie seu domínio sobre os assuntos da Semana 02:

Pergunta de Autoavaliação	Preciso rever	Compreendi	Domino com segurança
Sei explicar por que componentizamos uma tela?			
Sei passar e desestruturar props em um componente?			
Entendo por que não devo alterar o valor de uma prop recebida?			
Sei a diferença entre `onClick={fn}` e `onClick={() => fn(param)}`?			
Sei iterar um vetor com `.map()` e aplicar a prop `key` estável?			
Sei exibir conteúdos condicionais com operador ternário ou `&&`?			
Consigo criar layouts que se adaptam a celulares e computadores?			

---

9. Glossário técnico do estudante

- **Callback:** Função que você passa para outro componente para ser executada quando algo acontecer (ex.: ao clicar em um botão).
- **CamelCase:** Estilo de escrita como `onClick` ou `handleAddTask` (primeira letra minúscula, próximas palavras com maiúscula).

- **Desestruturação:** Forma rápida do JavaScript de extrair dados de objetos: `const { title, priority } = props`.
- **Early Return:** Prática de verificar uma condição e retornar logo no início da função (ex: se a lista estiver vazia).
- **Key:** Identificador único obrigatório que ajuda o React a saber quais itens de uma lista foram alterados ou removidos.
- **PascalCase:** Estilo de escrita onde todas as palavras iniciam com letra maiúscula (ex.: `TaskItem`, `TaskList`). É obrigatório no nome de componentes React!
- **Props:** Mecanismo do React para passar informações de um componente pai para um filho.
- **Virtual DOM:** Cópia leve da árvore da interface em memória, usada pelo React para calcular as atualizações mínimas necessárias no navegador.
